NavCore RX — Platform Guide

> One tested STM32G431 board with GPS, 9-axis IMU, LoRa 900MHz, and motor/servo control.
> Drop it into any RC vehicle. Change two functions. Ship.

Hardware at a glance

Component	**Part**	**What it gives you**
MCU	STM32G431CBT6	170 MHz Cortex-M4, all firmware runs here
IMU	ICM-20948	Roll, pitch, yaw, accel, gyro, temp
Compass	AK09916 (inside ICM)	Tilt-compensated magnetic heading
GPS	LC86GLAMD	Position, speed, course, HDOP via UART
Radio	E220-900T22D	LoRa 900 MHz, 5 km+ range

Firmware structure

The firmware is split into two layers. You only touch the top one.

```
┌─────────────────────────────────────────────────────┐
│  APPLICATION LAYER  ←  you edit this               │
│  IMU_CheckSafety()  ·  Servo_Step()  ·  config.h   │
└─────────────────────────────────────────────────────┘
                        │ calls
┌───────────────────────▼─────────────────────────────┐
│  CORE LIBRARY  ←  provided, don't change            │
│  IMU_ReadAll()  GPS_Poll()  E220_Send()             │
│  Motor_Set()  ReadBattery()  FilterDistance()         │
└─────────────────────────────────────────────────────┘
                        │ runs on
┌───────────────────────▼─────────────────────────────┐
│  HARDWARE                                           │
│  STM32G431  ·  ICM-20948  ·  LC86G  ·  E220        │
└─────────────────────────────────────────────────────┘
```

What the core library gives you

Call these from your application — no driver knowledge needed:

```
// Sensors
float roll    = imu_get_roll();       // degrees, from ICM-20948
float pitch   = imu_get_pitch();
```

```
float heading = imu_get_heading();    // 0–360°, tilt-compensated
float lat     = gps_get_lat();        // decimal degrees
float speed   = gps_get_speed();      // km/h

// Radio (from TX joystick)
int throttle  = radio_get_ch(1);      // 0–100
int steering  = radio_get_ch(2);      // −100 to +100

// Outputs
motor_set(1, throttle);               // 0–100% → calibrated ESC pulse
servo_set(1, position);               // pulse in µs
```

Adapting for your application

The only functions you need to change:

What to change	Where	Why
Safety/tilt logic	`IMU_CheckSafety()`	Each vehicle has different tilt limits
Servo positions	`servo_positions[]` in `Servo_Step()`	Bait drop, rudder, pump, gimbal
Motor start pulses	`MOTOR_LEFT_START` / `MOTOR_RIGHT_START` in `config.h`	Each ESC arms differently
Update intervals	`GPS_SEND_INTERVAL`, `IMU_READ_INTERVAL`	Match your vehicle's timing needs
Radio channels	`radio_get_ch(n)` mapping in main loop	Map joystick axes to your outputs

By application

Hovercraft / RC boat

- Keep: GPS, motors, radio, battery, IMU tilt
- Change: `IMU_CheckSafety()` thresholds for your hull geometry
- Change: `Servo_Step()` → rudder PWM sweep instead of bait drop

Balancing robot

- Keep: `IMU_ReadAll()`, `imu_pitch`, `imu_roll`, radio, motors
- Remove: GPS, home averaging, distance filter
- Add: PID control loop in main loop using `imu_pitch`
- Change: motor PWM range to allow reverse direction

Drone (attitude control)

- Keep: `IMU_ReadAll()`, GPS, radio

- Change: `Motor_Set()` → 4× ESC outputs (TIM1 CH1–4)
- Change: radio protocol → 4-channel stick values
- Add: PID loops for roll/pitch/yaw, Madgwick or complementary filter

Crop sprayer / ground rover

- Keep: GPS, motors, radio, battery
- Remove: IMU tilt checks (flat terrain)
- Change: `Servo_Step()` → spray pump relay
- Add: `WP_GO` navigation using the GPS haversine math already in `CalcDistanceD()`

IMU — what works, what needs field setup

Feature	Status	Notes
Accel / gyro reading	Production ready	100 ms rate, 14-byte burst
Pitch / roll / tilt detection	Production ready	Tested in bait boat
Rough water detection	Production ready	Gyro-total threshold
Stuck detection	Production ready	Motor on + no movement
Magnetometer heading	⚠ Needs one-time calibration	Hard-iron offsets not yet measured
RTH navigation	Stub	Command reaches RX, PID not implemented
WP_GO autopilot	Stub	Coordinates received, autopilot not implemented

Mag calibration (one-time, 10 minutes):
Place the board away from large metal objects. Rotate it slowly through 360° on all three axes while logging `mag_raw[]`. Record min/max on each axis. Add the midpoint offsets to `IMU_Process()`:

```
mag_ut[0] = (float)(mag_raw[0] - offset_x) * 0.15f;
mag_ut[1] = (float)(mag_raw[1] - offset_y) * 0.15f;
mag_ut[2] = (float)(mag_raw[2] - offset_z) * 0.15f;
```

Do this once per installation. Motors and battery will cause interference — calibrate with everything powered on in the final mounting position.

Quick start

1. **Flash** the firmware to the RX board. Verify UART output shows `[IMU] OK` and `[RADIO] OK`.

2. **Set motor calibration** in `config.h` — adjust `MOTOR_LEFT_START` and `MOTOR_RIGHT_START` until motors spin at minimum throttle without stalling.
3. **Pick your application** — copy the closest example from `/examples/` into `/app/myapp.c`.
4. **Edit `IMU_CheckSafety()`** — replace the boat tilt conditions with your vehicle's safety requirements.
5. **Test radio link —** TX TFT should show battery%, satellite count, and GPS fix within 60 seconds of powering RX outdoors.

Reference application

The full production bait boat firmware (v5.5b, tested) lives in `/examples/bait_boat/`. Use it as the end-to-end reference for the TX↔RX protocol, GPS packet format, and how command retries work.

Files to change per application

```
your-application/
├── config.h             ← motor pulses, tilt limits, timing intervals
├── app/myapp.c          ← your control logic (copy from examples/)
└── (everything else)    ← leave unchanged
```